

Práctica de Laboratorio N° 11:

Implementación de Inserciones en Árboles Balanceados (AVL)

I. OBJETIVOS DE LA PRÁCTICA:

- Comprender el funcionamiento del algoritmo de inserción en un árbol AVL, identificando cuándo es necesario realizar rotaciones para mantener el balance del árbol.
- Implementar en lenguaje C++ el proceso de inserción de nodos en un árbol AVL, incluyendo las rotaciones simples y compuestas.
- Analizar los cambios estructurales en el árbol AVL tras cada inserción, mediante la representación gráfica del árbol en consola.

II. CONCEPTOS GENERALES:

Los árboles AVL son árboles binarios de búsqueda auto-balanceados. Su principal característica es que después de cada inserción o eliminación, el árbol se ajusta automáticamente para mantener su equilibrio, es decir, la diferencia de altura entre los subárboles izquierdo y derecho de cualquier nodo no debe superar 1. Cuando esta diferencia se rompe debido a una inserción, se debe **reestructurar el árbol mediante rotaciones** para devolverlo a un estado balanceado.

Restructuration del árbol balanceado:

Reestructurar el árbol significa **rotar los nodos del mismo** para llevarlo a un estado de equilibrio. La rotación puede ser **simple** o **compuesta**:

- **Rotación simple:** afecta solo a **dos nodos** y se da en los siguientes casos:
 - **Rotación a la izquierda (DD):** ocurre cuando se inserta en la rama derecha del subárbol derecho.
 - **Rotación a la derecha (II):** ocurre cuando se inserta en la rama izquierda del subárbol izquierdo.
- **Rotación compuesta:** afecta a **tres nodos** y se da en los siguientes casos:
 - **Rotación derecha-izquierda (DI):** ocurre cuando se inserta en la rama izquierda del subárbol derecho.
 - **Rotación izquierda-derecha (ID):** ocurre cuando se inserta en la rama derecha del subárbol izquierdo.

III. DESARROLLO DE LA PRÁCTICA:

A partir del código base de árboles binarios de búsqueda desarrollados en las prácticas anteriores, realice lo siguiente:

1. Modifique la clase Nodo

Agregue un atributo adicional llamado *fe* (factor de equilibrio), necesario para mantener el balance del árbol AVL. Este atributo debe ser un número entero que toma los valores -1, 0 o 1 según el balance del subárbol en cada nodo.

```
5 class Nodo
6 {
7 public:
8     int info;
9     int fe; // factor de equilibrio
10    Nodo* izq;
11    Nodo* der;
12    Nodo(int valor) {
13        info = valor;
14        fe = 0;
15        izq = der = NULL;
16    }
17 };
18
```

2. Implemente el método **inserta_balanceado**

Cree el método **inserta_balanceado** tomando en cuenta el pseudocódigo revisado en clase.

void inserta_balanceado(Nodo*& apnodo, bool& bo, int dato)

3. Análisis de inserciones:

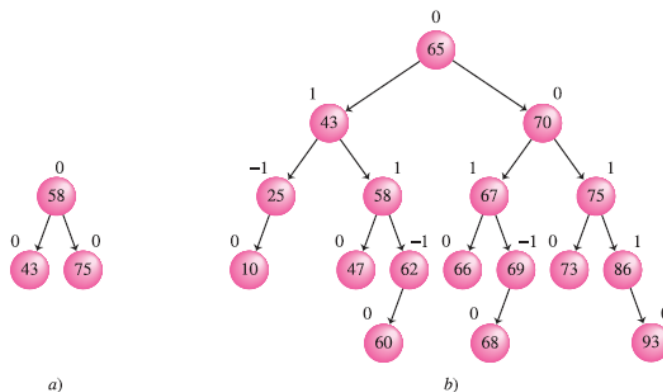
Dado como dato el árbol balanceado de la figura a), verifique si el mismo queda igual al de la figura b)

luego de insertar las siguientes claves:

86 - 65 - 70 - 67 - 73 - 93 - 69 - 25 - 66 - 68 - 47 - 62 - 10 - 60

Para cada inserción:

- Muestre el árbol resultante
- Indique si se realizó una rotación (simple o doble) y cuál fue.
- Justifique la necesidad de la reestructuración con base en el factor de equilibrio.
- Finalmente, muestre el **recorrido en Inorden** del árbol resultante y verifique que esté ordenado ascendentemente.



Ejemplo de Análisis:

Inserción: 86 Árbol:

86

No se realiza rotación.

Inserción: 65 Árbol:

86

/

65

No se realiza rotación.

Inserción: 70 Árbol:

86

/

65

\

70

Factor de equilibrio de 86: -2 → **Rotación Izquierda-Derecha (ID)**.

Árbol tras rotación:

70

/ \

65 86

Repetir este análisis para los valores siguientes.



4. Inserte la secuencia 15 - 20 - 24 - 10 - 13 - 7 - 30 - 36 - 25 en:
- un Árbol Binario de Búsqueda (ABB) sin balanceo
 - un Árbol AVL con reestructuración
- a) Muestre las estructuras finales de ambos árboles.
 - b) Compare la altura de ambos árboles.
 - c) Explique por qué el AVL es más eficiente en búsquedas en este caso.